

Quiz 3

Q1:

```
db.myCollection.find({
  $and: [
    { age: { $gt: 25 } },
    {
      $or: [
        { profession: "Software Engineer" },
        { profession: "Data Scientist" }
      ]
    }
  ]
})
```

This query will retrieve documents from `myCollection` where:

- The `age` field is greater than 25.
- The `profession` field is either "Software Engineer" or "Data Scientist".

Q2:

```
db.peaks.find({
  $and: [
    { location: { $all: ["China", "Nepal"] } },
    { height: { $gt: 8000 } },
    { "details.prominence": { $gt: 1000 } }
  ]
})
```

- `{ location: { $all: ["China", "Nepal"] } }` ensures that the `location` array contains both "China" and "Nepal", regardless of the order.
- `{ height: { $gt: 8000 } }` ensures that the `height` field is greater than 8000 meters.
- `{ "details.prominence": { $gt: 1000 } }` ensures that the `prominence` field within the nested `details` document is greater than 1000 meters.

Q3:

```
db.oceanicRecords.find(
  {},
  {
    _id: 0,
    species: 1,
    depth: 1,
```

```

    "observations.lastSeen": 1,
    "observations.sightingsCount": 1
  }
)

```

Here's a breakdown of the query:

- The first parameter `{}` is the query filter, which is empty in this case, meaning it will match all documents in the `oceanicRecords` collection.
- The second parameter specifies the projection, indicating which fields to include or exclude in the results:
 - `_id: 0` excludes the `_id` field from the results.
 - `species: 1` includes the `species` field.
 - `depth: 1` includes the `depth` field.
 - `"observations.lastSeen": 1` includes the `lastSeen` field from the nested `observations` document.
 - `"observations.sightingsCount": 1` includes the `sightingsCount` field from the nested `observations` document.

Q4:

```

db.cityEvents.aggregate([
  {
    $match: {
      $and: [
        { city: { $in: ["New York", "Los Angeles"] } },
        { attendees: { $gt: 100 } }
      ]
    }
  },
  {
    $sort: { date: -1 }
  },
  {
    $limit: 5
  },
  {
    $project: {
      _id: 0,
      eventName: 1,
      city: 1
    }
  }
])

```

Q5:

```
db.products.aggregate([
  {
    $group: {
      _id: "$category",
      count: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      category: "$_id",
      count: 1
    }
  }
])
```

Q6:

a:

People continue to use MongoDB and similar NoSQL databases because of their architectural flexibility, allowing for dynamic and evolving data models without the need for predefined structures. NoSQL databases also offer horizontal scalability, making them ideal for handling large amounts of data and high traffic by distributing the load across multiple servers. They are optimized for performance, providing low latency and high throughput for read and write operations, which is critical for real-time applications. Additionally, NoSQL databases are well-suited for storing unstructured or semi-structured data such as JSON and XML, which makes them adaptable to a variety of data types. Ease of development and fast iteration cycles further enhance their appeal, especially for rapidly changing requirements. Specific use cases such as real-time analytics, content management, and recommendation engines benefit greatly from the capabilities of NoSQL.

b:

Due to the maturity and stability of SQL Database, some companies are returning to SQL Database, which provides strong transaction support (ACID properties) to ensure data consistency and reliability, which is critical for mission-critical applications. SQL databases excel at handling complex queries and analytics, thanks to the powerful SQL language that facilitates data manipulation and retrieval. They enforce strict schema definitions and constraints, maintaining data integrity and consistency, which is critical for applications that require high data accuracy. SQL Database is further supported by a rich ecosystem of tools for database administration, monitoring and optimization. Standardizing SQL skills across different systems shortens the developer learning curve. Additionally, modern SQL databases like PostgreSQL now support unstructured data through features like JSONB, providing a hybrid approach that combines the benefits of SQL and NoSQL paradigms.